Orbit closure for iPRA

Exact equivalence, successor uniqueness,
and the remaining complexity gap

Prepared for Vladimir Reshetnikov

12 November 2026

Abstract

This report does not establish an axiomatization below the current complete-checker frontier. It gives a precise equivalent presentation of full first-order intuitionistic primitive recursive arithmetic, obtained by turning scalar recursion uniqueness into a reusable universal orbit equation. The target term disappears from this inference; the two term templates can be inferred from its conclusion. We also simplify the retraction argument: uniqueness for the fixed successor update alone implies contextual induction for every zero equation. Full arithmetic from this restricted principle remains unproved; the missing step is actual-equality reflection. A reproducible audit of the pinned collection corrects the compact coordinate envelope to 256 significant Python tokens, 276 AST nodes, and 506 normalized bytes, attained by different programs. We distinguish this envelope from a proved lower bound and distinguish compact public-language contracts from explicit syntax validation. All new mathematical results are written proofs; no new Lean or Rocq verification is claimed.

Contents

1 Outcome and comparison contract **2**

2 A complete ordinary axiomatization **2**

3 Exact equivalence with scalar uniqueness **4**

4 Why the ordinary theory is bi-interpretable with iPRA **4**

5 A narrower arithmetic principle: successor uniqueness **6**

6 Complete checker and executable evidence **8**

7 Audit of the current frontier **10**

8 Further research questions **10**

9 Claim boundaries **11**

A Inherited proof dependencies and reproducibility **11**

B Complete selected checker **12**

1 Outcome and comparison contract

The requested conjunction has three parts: a simpler complete presentation, bi-interpretability with full first-order iPRA, and a complete checker below the existing complexity envelope. The construction below establishes the second part and removes a certificate parameter from one presentation. It does *not* establish the requested overall size improvement. This distinction is part of the result, not a claim deferred to an unimplemented optimization.

All repository comparisons are pinned to commit

`62396b06020f794fa6dc01b4025a6a5db9d4d28a`.

The linked project lists ninety research packages and a Lean development [1]. Several attractive proposals are already present: witnessed and witness-free introductions, clause contexts, one-body contexts, inferred discharge witnesses, full computation normalization, and fused evaluation/discharge. They are not claimed as new here.

The complexity coordinates are the significant tokens, all Python AST nodes, and UTF-8 bytes after removal of actual docstrings, `ast.unparse` normalization, and one terminal LF, under CPython 3.12.14. Comments and layout tokens are excluded from the token count. Imports, input parsing, checking, and the success signal remain part of a complete program. Different programs attaining different coordinate minima are not one hypothetical program.

Coordinate	Value	A pinned witness
Significant tokens	256	`kernel_refinement`, token profile
AST nodes	276	`deterministic_opening`, compact AST profile
Normalized bytes	506	`deterministic_introduction`, optional compact profile

These are *attained upper bounds* on minimum implementation size, not theorems that no smaller implementation exists. The last two are easy to overlook if one reads only the top-level package table. The compact profiles operate at their documented canonical guarded public-language interface; they do not all validate the complete public grammar or every JSON field. Section 6 records a separate matched comparison for the new rule.

2 A complete ordinary axiomatization

2.1 Finite syntax and numerical domain

There is one numerical sort. Terms are

$$t ::= x \mid 0 \mid S(t) \mid \mathsf{I}(n, a; u.h).$$

Only u in the body h is bound by the iterator constructor; the counter n, initializer a, and passive parameters of h are outside that binding. All substitutions avoid capture. This is finite binding syntax, or equivalently a staged first-order signature with one function symbol $I_h(n, a, \vec{p})$ for each fixed earlier-stage body $h(u, \vec{p})$. There are no variables ranging over programs and no uniform total evaluator for all primitive-recursive descriptions.

Formulas use actual numerical equality, $\bot$, $\wedge$, $\vee$, $\to$, $\forall$, and $\exists$. Negation is $\neg A := (A \to \bot)$. Quantifiers range over the entire numerical domain.

2.2 Logical axioms and inference rules

Use single-conclusion intuitionistic natural deduction. A context is a finite list of retained assumptions. The following specifies all logical rules; common contexts in a rule can be obtained by weakening.

Rule	Premises and conclusion
Assumption	$A \in \Gamma$ gives $\Gamma \vdash A$.
Structural	Exchange, weakening, and contraction of retained assumptions are allowed. A proved formula may replace an assumed occurrence by substitution of its derivation (cut).
Conjunction	From $\Gamma \vdash A$ and $\Gamma \vdash B$, infer $\Gamma \vdash A \wedge B$; either conjunct follows from $\Gamma \vdash A \wedge B$.
Disjunction	Either disjunct gives $\Gamma \vdash A \vee B$. From that judgment, $\Gamma, A \vdash C$, and $\Gamma, B \vdash C$, infer $\Gamma \vdash C$.
Implication	From $\Gamma, A \vdash B$, infer $\Gamma \vdash A \to B$. From $\Gamma \vdash A \to B$ and $\Gamma \vdash A$, infer $\Gamma \vdash B$.
Universal	From $\Gamma \vdash A(z)$, with z fresh for Γ and all passive parameters of A, infer $\Gamma \vdash \forall x A(x)$. From $\Gamma \vdash \forall x A(x)$ infer $\Gamma \vdash A(t)$ for any term free for x.
Existential	From $\Gamma \vdash A(t)$ infer $\Gamma \vdash \exists x A(x)$. From that existential and $\Gamma, A(z) \vdash C$, with z fresh for Γ, C and all passive parameters of A, infer $\Gamma \vdash C$.
Falsity	From $\Gamma \vdash \bot$ infer any $\Gamma \vdash A$.
Equality	$\Gamma \vdash t = t$. From $\Gamma \vdash s = t$ and $\Gamma \vdash F(s)$ infer $\Gamma \vdash F(t)$, for any capture-avoiding formula template F.

No excluded-middle scheme, double-negation elimination, choice principle, or induction on arbitrary formulas is included.

2.3 All nonlogical axioms

Take the universal closures of the following schemes:

$$\mathsf{I}(0, a; u.h) = a, \tag{C0}$$

$$\mathsf{I}(Sn, a; u.h) = h[\mathsf{I}(n, a; u.h)/u], \tag{CS}$$

$$\forall n\,(n = 0 \vee \exists m\ n = Sm), \tag{Cov}$$

$$(0 = S0) \to \bot. \tag{Sep}$$

The separation axiom is explicit. Positive computation equations alone have a singleton model, so separation must not be silently inferred from them.

The only additional arithmetic inference is

$$\boxed{\frac{\Gamma \vdash f(Sz) = h[f(z)/u]}{\Gamma \vdash \forall y\big(f(y) = \mathsf{I}(y, f(0); u.h)\big)}} \quad \text{(Orbit)}.$$

Here $f(v, \vec{p})$ and $h(u, \vec{p})$ are fixed term templates. The update h has no free occurrence of the varying counter v or the eigenvariable z. The eigenvariable is absent from Γ and all passive parameters. Its only occurrences in the premise come from the displayed substitution. There is no extra induction hypothesis in the premise and no base premise: the initializer is inferred as $f(0)$.

Call this theory T_{Orb}. This is an explicit axiomatization, not an abbreviation for "all true primitive-recursive equations."

3 Exact equivalence with scalar uniqueness

For comparison, the repository's scalar-uniqueness theory T_{UR} has the same logic and axioms, but uses

$$\frac{\Gamma \vdash f(Sz) = h[f(z)/u]}{\Gamma \vdash f(t) = \mathsf{I}(t, f(0); u.h)} \quad \text{(UR)},$$

with z additionally fresh for the target t. Its completeness relative to equation induction is the result of `scalar_uniqueness` [2].

Theorem 3.1 (All open consequences are preserved)**.** *For every first-order context* Γ *and formula* A*,*

$$\Gamma \vdash_{T_{\mathrm{Orb}}} A \quad \Longleftrightarrow \quad \Gamma \vdash_{T_{\mathrm{UR}}} A.$$

Proof. An application of UR is Orbit followed by universal elimination at t. Conversely, choose a new variable w absent from the context and all templates and parameters. UR with target w gives $f(w) = \mathsf{I}(w, f(0); u.h)$. Universal introduction gives precisely the Orbit conclusion. These replacements work under arbitrary retained contexts and inside larger derivations. They are finite syntactic proof transformations. □

3.1 The inference as a reusable clause

A clause $[\,; y.E(y)]$ means the unconditional universal formula $\forall y E(y)$. In a clause calculus, Orbit appends

$$[\,; y.f(y) = \mathsf{I}(y, f(0); u.h)].$$

Clause Use then chooses an instantiation term when it is actually needed. The inference itself has no target argument. This is a reduction of certificate data, not removal of the mathematical uniqueness principle.

In the repository's native syntax, write

$$Q_y B := B(0) \to \forall y B(y), \qquad D_{x,y} B := \exists x \big(B(x, 0) \to \forall y B(x, y)\big).$$

Let $E(y)$ be the displayed orbit equation. Its base is a theorem by C0 and equality, so $Q_y E$ and $\forall y E$ are interderivable. Orbit can therefore introduce $Q_y E$ directly. If the proposed conclusion is supplied as a raw syntax tree, its left term supplies f, its right iterator supplies h, and its initializer is checked against the zero instance $f(0)$.

Remark 3.2 (An essential check)**.** The update must be independent of the bound counter. If this condition is omitted, the apparent instance $f(y) = y$, $h(y, u) = Sy$ has the reflexive premise $Sz = Sz$, but would assert $n = \mathsf{I}(n, 0; u.Sn)$. At $n = 2$ the right side is 3. A checker that omits this independence check proves a false equation. The delivered negative fixture exercises this exact failure mode.

4 Why the ordinary theory is bi-interpretable with iPRA

We use full first-order intuitionistic PRA: intuitionistic equality logic, symbols for all fixed primitive-recursive descriptions and their defining equations, the successor axioms, and induction for quantifier-free formulas. We do not identify it with its equational or universal fragment.

Let T_{eq} replace Orbit by contextual literal-equation induction:

$$\frac{\Gamma \vdash s(0) = r(0) \qquad \Gamma, s(z) = r(z) \vdash s(Sz) = r(Sz)}{\Gamma \vdash s(t) = r(t)},$$

with the usual freshness condition on z. The repository establishes T_{UR} and T_{eq} have exactly the same consequences [2]. This is the substantive inherited theorem; mere agreement on the standard natural numbers would not suffice.

4.1 The inherited theorem's dependency order

For clarity, the proof does not assume a general equality test or pairing at the start. It proceeds as follows.

1. Scalar uniqueness constructs a finite predecessor term P and proves $P0 = 0$ and $P(Sn) = n$. Thus successor injectivity is derived.
2. With $d(a, b) = \mathsf{I}(b, a; u.Pu)$, it proves the monus cancellation laws. A diagonal descent argument for Boolean terms constructs an explicit scalar accumulator and proves
$$a + d(b, a) = b + d(a, b).$$
3. Consequently the fixed numerical term
$$\Delta(a, b) = \mathrm{sg}\big(d(a, b) + d(b, a)\big), \qquad \mathrm{sg}(v) = \mathsf{I}(v, 0; u.S0),$$
is Boolean and satisfies $\Delta(a, b) = 0 \leftrightarrow a = b$ for actual equality on the whole domain.
4. The retraction argument gives induction for zero equations. Applying it to $\Delta(s(n), r(n)) = 0$ recovers all literal-equation induction.

The reverse implication is short: equation induction on $f(n) = \mathsf{I}(n, f(0); u.h)$ proves UR, using its retained recurrence premise and the two computations. Appendix A includes the precise inherited proof fragments with provenance; their contents are not claimed as new.

4.2 Interpretation into iPRA

Interpret each fixed iterator by primitive recursion that ignores its counter argument in the update. The domain formula is $x = x$, and equality remains equality. The computation axioms hold by definition. Coverage, separation, and literal-equation induction hold in iPRA. Induction proves UR as just described, so it proves Orbit as well. All ordinary logical symbols are translated identically.

4.3 Interpretation of iPRA into the orbit theory

After actual-equality reflection is available, the selector
$$C(v, a, b) := \mathsf{I}(v, a; u.b)$$
compiles every quantifier-free formula into a Boolean characteristic term. This includes implication, because its quantifier-free operands are decidable. Finite Boolean truth tables convert each quantifier-free induction instance to an equation-induction instance. No decidability principle for quantified formulas follows or is assumed.

The scalar arithmetic bridge constructs pairing π and projections L, H with
$$L(\pi(a, b)) = a, \qquad H(\pi(a, b)) = b.$$
These are internal equations over the original numerical domain, not assertions restricted to standard numerals. One explicit implementation uses $\pi(a, b) = (a + b)^2 + a$, a bounded square-root search, and truncated subtraction; the search is implemented by a saturating scalar iterator. Quantifier-free induction, now available, proves its bounds and the displayed inverse equations [3].

For a primitive recursion

$$f(0,\vec{p}) = g(\vec{p}), \qquad f(Sn,\vec{p}) = h(n, f(n,\vec{p}),\vec{p}),$$

define its translated state by

$$V(n,\vec{p}) = \mathsf{I}(n, \pi(0, g^*(\vec{p})); u.\pi(SL(u), h^*(L(u), H(u), \vec{p}))), \qquad f^*(n,\vec{p}) = H(V(n,\vec{p})).$$

Equation induction proves $L(V(n,\vec{p})) = n$. The projection equations then prove both defining equations of f^*. This is external structural recursion on each finite PR description. It does not postulate a numerical function that evaluates every PR description uniformly.

Theorem 4.1 (Bi-interpretation and equiconsistency)**.** *The ordinary theory T_{Orb} and full first-order* iPRA *are bi-interpretable. Both numerical domains are whole, equality is actual equality, and the graph of each numerical composite isomorphism is $x = y$. They are equiconsistent.*

Proof. The preceding constructions interpret every axiom and rule in both directions. To verify the composites, induct externally on finite operation descriptions. A double-translated iterator has the original initializer and pointwise the original update after the lower-level descriptions have been identified. Equation induction proves uniqueness of the recurrence, so its values equal the original values at every counter. The same induction proves uniqueness for each ordinary primitive-recursive definition, including its counter argument.

The domain and equality are unchanged. Hence $x = y$ is internally total, one-to-one, onto, and preserves every translated operation. Ordinary logical symbols are unchanged, so both formula composites follow by structural induction on formulas. Effective translations carry finite proofs of $\bot$ in either direction. This is equiconsistency; it is not a proof by either theory of its own consistency. □

5 A narrower arithmetic principle: successor uniqueness

Replace UR by its fixed-successor restriction

$$\frac{\Gamma \vdash f(Sz) = S(f(z))}{\Gamma \vdash f(t) = \mathsf{I}(t, f(0); u.Su)} \quad \text{(SU)}.$$

This theory, T_{SU}, retains the ordinary logic and C0, CS, Cov, Sep. It has no supplied update template. No full-iPRA equivalence is claimed for it.

Lemma 5.1 (The identity orbit)**.** *T_{SU} proves $\mathsf{I}(t, 0; u.Su) = t$.*

Proof. Use SU with $f(z) = z$. Its premise is reflexivity, and reverse the resulting equality. □

Lemma 5.2 (Retraction rigidity using successor uniqueness only)**.** *Suppose, in a fixed retained context Γ, a term template e satisfies*

$$e(0) = 0, \qquad e(e(x)) = e(x), \qquad e(S(e(x))) = S(e(x))$$

at a fresh arbitrary x. Then $\Gamma \vdash e(t) = t$ for every term t.

Proof. Put $h(u) = e(S(e(u)))$ and $r(n) = \mathsf{I}(n, 0; u.h)$. We first prove

$$e(r(n)) = r(n)$$

by the zero/successor coverage axiom, with no induction and no arbitrary-update uniqueness. At zero it is $e(0) = 0$. If $n = Sm$, then

$$e(r(Sm)) = e(h(r(m))) = h(r(m)) = r(Sm),$$

because $h(r(m))$ is itself an e-image and e is idempotent. Therefore

$$r(Sn) = e(S(e(r(n)))) = S(e(r(n))) = S(r(n)).$$

SU gives $r(n) = \mathsf{I}(n, 0; u.Su) = n$. Substituting into the first displayed equality gives $e(n) = n$, and specialization gives the target instance. □

The simplification is the first step of this proof: the existing retraction proof uses arbitrary-update UR to show that e fixes the orbit. Since every successor value of that orbit is already in the image of e, ordinary numerical cases suffice.

Theorem 5.3 (Contextual zero-equation induction)**.** *In T_{SU}, for every fixed scalar term $q(n, \vec{p})$, the rule*

$$\frac{\Gamma \vdash q(0) = 0 \qquad \Gamma, q(z) = 0 \vdash q(Sz) = 0}{\Gamma \vdash q(t) = 0}$$

is derivable, with z fresh for the retained context, target, and passive parameters.

Proof. First, no successor is zero. If $Sv = 0$, applying the term $w \mapsto C(w, 0, S0)$ and using computation gives $S0 = 0$, contradicting Sep. Thus coverage makes $A(n) :\equiv q(n) = 0$ decidable, intuitionistically.

Define $e(n) = C(q(n), n, 0)$. Under $A(n)$ we have $e(n) = n$; under $\neg A(n)$, coverage of $q(n)$ gives $e(n) = 0$. The assumed base therefore gives $A(e(n))$ in both cases. Applying the true-case identity at $e(n)$ gives idempotence $e(e(n)) = e(n)$. The retained step, specialized at $e(n)$, gives $A(S(e(n)))$ and hence $e(S(e(n))) = S(e(n))$. The base also gives $e(0) = 0$. Retraction rigidity yields $e(n) = n$; transport in A gives $A(n)$.

The case split was only on a numerical zero test. The retained context may contain arbitrary quantified formulas; its decidability is not needed. □

5.1 The exact unproved step

The theorem does not justify replacing UR by SU in the complete axiomatization. To recover arbitrary equation induction by the usual route, one needs a term Δ satisfying

$$\Delta(a, b) = 0 \quad \Longleftrightarrow \quad a = b$$

for actual equality on the whole domain. The scalar-UR proof of this statement uses updates other than successor, as does its predecessor construction. Those results cannot simply be reused inside T_{SU}.

The existing two-fiber countermodel shows that zero-equation induction, even with successor injectivity, coverage, and all scalar computations, does not imply arbitrary equation induction [4]. It is *not* a countermodel to SU. In fact it refutes SU: fix a nonstandard counter p and let $e(a) = \mathsf{I}(p, a; u.u)$. Its specified identity-iterator choice makes e the projection to the label-zero point of each fiber. Hence $e(0) = 0$ and $e(Sa) = S(e(a))$, but e is not the identity. SU and the identity-orbit lemma would force $e(a) = a$. Thus both the inference "zero induction is enough" and the inference "SU is incomplete by that model" would be unjustified.

5.2 A precise intermediate target: additive cancellation

Proposition 5.4. *Define $a + b := \mathsf{I}(b, a; u.Su)$. The theory T_{SU} proves the commutative-monoid equations for addition. If $\Gamma \vdash k(Sz) = k(z)$ for a term template k at a fresh variable, then*

$$\Gamma \vdash n + k(n) = n + k(0).$$

Consequently, additive cancellation would imply uniqueness for the identity update.

Proof. Computation gives $a + 0 = a$ and $a + Sb = S(a + b)$; the identity-orbit lemma gives $0 + b = b$. The term $f(n) = S(a + n)$ has initial value Sa and successor update, so SU gives $S(a + n) = Sa + n$. Therefore $f(n) = n + a$ has initial value a and successor update; SU gives $n + a = a + n$. Similarly, $f(n) = a + (b + n)$ has initial value $a + b$ and successor update, giving associativity.

Now $f(n) = n + k(n)$ has initial value $k(0)$, and

$$f(Sn) = Sn + k(Sn) = Sn + k(n) = S(n + k(n)) = S(f(n)).$$

SU gives $f(n) = k(0) + n = n + k(0)$. If cancellation were available, cancelling n would yield $k(n) = k(0)$. No cancellation is assumed in the proposition. □

6 Complete checker and executable evidence

6.1 An explicit native proof presentation

The delivered program uses the raw native language $A ::= (s = t) \mid Q_y A \mid D_{x,y} A$, with the ordinary readings in Section 3. It replaces only the UR instruction of the inherited strict checker. Its full rule inventory is:

Rule	Premises and conclusion
Hyp	$A \in \Gamma$ gives $\Gamma \vdash A$.
C0, CS	The two displayed iterator computation equations.
Tr	From $s = t$ and $h(s) = u$ infer $u = h(t)$, in the same context.
QE	From $Q_y B$ and $B(0)$ infer $B(t)$.
DE	From $D_{x,y} B$ and $\Gamma, Q_y B(b, y) \vdash A$ infer $\Gamma \vdash A$, with b fresh for Γ, A and all passive parameters of B.
DI	From $\Gamma, B(t, 0) \vdash B(t, Sz)$ infer $\Gamma \vdash D_{x,y} B$, with z fresh for Γ, B, t.
Orbit	From $\Gamma \vdash f(Sz) = h(f(z))$ infer $\Gamma \vdash Q_y(f(y) = \mathsf{I}(y, f(0); u.h))$, with the stated freshness and counter-independence conditions.

There is no primitive progress rule or general induction rule in this list. Reflexivity and the usual equality rules are derived from computation and crossed transport. The native logical reconstruction is inherited from the scalar-UR presentation, and the exact UR/Orbit translations preserve it.

In the native direction from UR to Orbit, obtain the successor instances of the orbit equation by UR, form D with an unused witness by DI by cases, and eliminate that D to obtain the Q closure. In the opposite direction, use QE; its base premise is the symmetric C0 equation. This proves exact native open-consequence equivalence without assuming a primitive universal quantifier in the checker.

6.2 Public input and all trusted work

Each input is a JSON object with exactly four keys: `variables`, `context`, `goal`, and `proof`. The numerical scope is an exact nonnegative integer. The context and goal are explicitly validated. Terms use tags `v,z,s,i`, formulas use `=,q,d`, and proofs use the rule tags just listed. Numerical indices are de Bruijn indices, with zero referring to the nearest binder. Only an iterator's update body binds a numerical variable. Q binds one variable; D binds its closure variable at index zero and its witness at index one.

The selected instruction is

```
['orbit', ['q', ['=', f, ['i', ['v', 0], a, H]]], premise]
```

It carries a proposed closure and one premise proof. The checker reads f and the update from the closure, verifies that a is literally the shifted zero instance of f, removes the counter coordinate from the update, and rejects any remaining dependence on that coordinate. It then checks the recurrence premise in the enlarged numerical scope with only the shifted retained assumptions. An induction hypothesis is never added.

The counter-removal check uses the existing capture-avoiding substitution function with an out-of-scope sentinel. A temporary Q wrapper protects the update's state variable. Any occurrence of the forbidden counter, including one beneath a nested iterator, becomes the sentinel; the term validator then rejects it. Legitimate passive parameters simply lose the removed coordinate.

Acceptance is normal termination with exact output `ACCEPT` and a terminal newline. Exceptions reject. The implementation does not use Python assertions, so `python -O` does not remove checks. As with the comparison programs, completeness is stated for sufficiently resourced execution of each finite proof; a fixed implementation recursion limit is not an arithmetic axiom.

6.3 Matched measurements

Complete strict program	Tokens	AST nodes	Bytes
Inherited scalar UR control	1124	1175	3393
Orbit, explicit templates, no target	1132	1178	3400
Orbit, templates inferred from closure	1145	1194	3455

The selected implementation adds 62 normalized bytes relative to its matched control. Reading templates from a conclusion removes certificate fields but adds structural and scope checks. This is direct evidence that the interface theorem is not yet a proof-checker simplification under the requested metric.

6.4 Proof translation and tests

`orbit/translate_ur.py` recursively translates arbitrary finite scalar-UR proof trees to either Orbit format. At each UR inference it constructs the Orbit closure, a proof of its zero instance, and QE at the old target. Other rule tags are preserved and premise proofs are translated recursively. This translator is untrusted: its output is always submitted to the standalone target checker.

The recorded validation includes 156 checker-process tests under ordinary and optimized Python: 92 expected acceptances and 64 expected rejections, with zero mismatches. Two actual command-line translator invocations were followed by independent target-checker processes and accepted. A separate inherited suite contains 66 process checks. Cases cover open scopes, retained contexts, nested iterators, DI and DE, forbidden counter dependence, a wrong initializer, attempted use

of a nonexistent induction hypothesis, Boolean indices, and escaping variables. Counts refer to executions, not to distinct mathematical theorems.

These tests verify concrete implementation behavior. The universal equivalence argument is the mathematical proof above; no finite test suite replaces it.

7 Audit of the current frontier

The audit retrieves the ninety package trees recursively and records that no tree response is truncated. It inventories all 5,825 Python paths under `Research/`, including nested reference and evidence copies. They represent 1,380 distinct Git blobs. Every blob is included in the archive, its exact Git object hash is checked, and its source is parsed and measured under CPython 3.12.14. The audit does not execute these source programs.

The three frontier witnesses are:

Existing file (under `Research/`)	Tokens	AST	Bytes
`kernel_refinement/code/checker_tokens.py`	256	283	594
`deterministic_opening/compact/code/ast.py`	263	276	580
`deterministic_introduction/implementation/code/compact_optional.py`	267	303	506

The last checker already contains the integer identity $\sim(-k) = k - 1$ as a two-byte shortening of the scope update. Rediscovering that change would not be a new improvement. Sources smaller in one isolated coordinate were inspected for their role: a report generator, measurement script, test harness, or script importing another local checker is not itself a complete standalone acceptance program. The audit metadata records those classification decisions.

The corpus covers every `.py` file under `Research/`. The frozen `docs/incoming/` tree contains only its empty placeholder, so there is no unfiled proposal archive there at this commit. The audit does not establish a lower bound on other programming languages, non-Python artifacts, every compressed file elsewhere, or future proposals. Nor does it formally reverify the mathematical soundness claims of every inherited checker.

The complete ledger, exact sources, and offline replay script are under `audit/`. The replay verifies source identities and measurements, not the semantic eligibility of arbitrary programs by an automated test. That eligibility distinction is explicit in the archived classification record.

8 Further research questions

1. Does SU derive actual-equality reflection in the scalar language? A positive proof must supply the predecessor and equality detector internally; a negative proof must construct a model satisfying every fixed-body computation and every contextual SU instance.
2. Does uniqueness for only the identity update, added to SU, suffice for full equation induction? The existing two-fiber obstruction specifically fails identity-update uniqueness, but excluding that example is not a completeness proof.
3. Can the target-free Orbit rule be combined with the smaller public-language encodings without adding more syntax recognition than it removes? Any claim should include the complete public validator or state the same canonical-input contract as its comparator.
4. Is there a complete presentation with a single program strictly below all three existing coordinate minima? Removing a printed success signal or embedding a compressed program changes the accounting question and does not supply a mathematical reduction.

5. Can the new retraction lemma and the Orbit equivalence be formalized in the existing Lean development with a substantially shorter dependency chain? This is a proof-engineering question distinct from numerical checker size.

9 Claim boundaries

The established new mathematics consists of the target-free Orbit interface and the reduction from SU to contextual zero-equation induction. The first preserves a complete theory; the second identifies a more restricted hypothesis that suffices for a particular proof, with full arithmetic strength unresolved. The general scalar-UR completeness theorem, the scalar compiler, and the native logical reconstruction are inherited and cited.

No absolute simplest axiomatization, proved global lower bound, new proof-assistant verification, or complete checker beating the current envelope is asserted. Finite numerical tests are implementation evidence; they do not verify statements in nonstandard arithmetic models.

References

[1] V. Reshetnikov, *ProveIt: Concise presentations of intuitionistic primitive recursive arithmetic*, frozen commit `62396b06020`. Project README.

[2] ProveIt, *Scalar iteration uniqueness*, `Research/scalar_uniqueness/ur_complete.tex`, with `ur_retraction.tex`, `ur_predecessor.tex`, and `ur_reflection.tex`, at the same commit. Pinned source.

[3] ProveIt, *Arithmetic strength and both ordinary interpretation composites*, in `Research/canonical_recursors/report.tex`, at the same commit; scalar pairing and primitive-recursion simulation. Pinned report source.

[4] ProveIt, *A model separating zero induction from equation induction*, in `Research/canonical_recursors/report.tex`, at the same commit. Pinned countermodel.

[5] ProveIt, `Research/kernel_refinement/code/checker_tokens.py`, at the same commit. Token comparator.

[6] ProveIt, `Research/deterministic_opening/compact/code/ast.py`, at the same commit. AST comparator.

[7] ProveIt, `Research/deterministic_introduction/implementation/code/compact_optional.py`, at the same commit. Byte comparator.

A Inherited proof dependencies and reproducibility

The archive's `reference/scalar_uniqueness/` contains the four unchanged TeX fragments used in Section 4, with their exact Git blob hashes recorded in the provenance file. They supply the scalar predecessor, the explicit accumulator for actual-equality reflection, and the retraction argument. The new SU proof in Section 5 replaces the first uniqueness use in that retraction argument. The report uses the inherited theorem at precisely the point indicated in Section 4; it does not relabel that theorem as new.

The source audit is reproducible offline from the archived source blobs and metadata. The report can be rebuilt with `build_pdf.py`; measurements and certificate regression checks are reproduced with `reproduce.py`. All checker code, validation scripts, metadata, and TeX are included in the ZIP.

B Complete selected checker

```
import json
import sys

def require(condition):
    if not condition:
        raise ValueError('Invalid certificate')

def term(a, n):
    match a:
        case ['v', i]:
            require(type(i) is int and 0 <= i < n)
        case ['z']:
            pass
        case ['s', b]:
            term(b, n)
        case ['i', c, b, h]:
            term(c, n)
            term(b, n)
            term(h, n + 1)
        case _:
            raise ValueError('Invalid term')

def formula(a, n):
    match a:
        case ['=', s, t]:
            term(s, n)
            term(t, n)
        case ['q', b]:
            formula(b, n + 1)
        case ['d', b]:
            formula(b, n + 2)
        case _:
            raise ValueError('Invalid formula')

def walk(a, f, d=0):
    if a[0] == 'v':
        return f(a[1], d)
    return [a[0]] + [walk(b, f, d + {'i': [0, 0, 1], 'q': [1], 'd': [2]}.get(a[0], [0] * len(a))[j]) for j, b
     in enumerate(a[1:])]

def shift(a, k=1):
    return walk(a, lambda i, d: ['v', i + k if i >= d else i])

def sub(a, t, drop=1):
    return walk(a, lambda i, d: ['v', i] if i < d else shift(t, d) if i == d else ['v', i - drop])

def infer(p, g, n):
    match p:
        case ['hyp', i]:
            require(type(i) is int and 0 <= i < len(g))
            return g[i]
        case ['zero', a, h]:
            term(a, n)
            term(h, n + 1)
            return ['=', ['i', ['z'], a, h], a]
        case ['succ', c, a, h]:
            term(c, n)
            term(a, n)
```

```
            term(h, n + 1)
            return ['=', ['i', ['s', c], a, h], sub(h, ['i', c, a, h])]
        case ['qe', t, p, q]:
            term(t, n)
            a = infer(p, g, n)
            require(a[0] == 'q')
            require(infer(q, g, n) == sub(a[1], ['z']))
            return sub(a[1], t)
        case ['de', a, p, q]:
            formula(a, n)
            b = infer(p, g, n)
            require(b[0] == 'd')
            require(infer(q, [['q', b[1]]] + [shift(x) for x in g], n + 1) == shift(a))
            return a
        case ['di', b, t, p]:
            formula(b, n + 2)
            term(t, n)
            a = sub(['q', b], t)[1]
            h = shift(sub(a, ['z']))
            require(infer(p, [h] + [shift(x) for x in g], n + 1) == sub(a, ['s', ['v', 0]], 0))
            return ['d', b]
        case ['orbit', ['q', ['=', f, ['i', ['v', 0], a, h]]] as b, p]:
            formula(b, n)
            h = sub(['q', h], ['v', n])[1]
            term(h, n + 1)
            require(a == shift(sub(f, ['z'])))
            require(infer(p, [shift(x) for x in g], n + 1) == ['=', sub(f, ['s', ['v', 0]], 0), sub(h, f, 0)])
            return b
        case ['tr', a, p, q]:
            term(a, n + 1)
            e = infer(p, g, n)
            f = infer(q, g, n)
            require(e[0] == f[0] == '=' and f[1] == sub(a, e[1]))
            return ['=', f[2], sub(a, e[2])]
        case _:
            raise ValueError('Invalid proof')

def check(c):
    require(type(c) is dict and set(c) == {'variables', 'context', 'goal', 'proof'})
    n, g, a, p = (c[k] for k in ['variables', 'context', 'goal', 'proof'])
    require(type(n) is int and n >= 0 and (type(g) is list))
    for b in g:
        formula(b, n)
    formula(a, n)
    require(infer(p, g, n) == a)
if __name__ == '__main__':
    try:
        check(json.load(sys.stdin))
    except (ValueError, TypeError, IndexError, KeyError, RecursionError):
        sys.exit(1)
    print('ACCEPT')
```